

Project- High Level Design On MedScribe AI

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Vinay Kuthey	EN22CS3011084
2	Viraj Agrawal	EN22CS3011089
3	Lakshita Singh	EN22CS3011135
4	Vishakha Shadija	EN22ME304114
5	Vishakha Hilsayan	EN22EL301065

Group Name: 09D10

Project Number: AAI-22

Industry Mentor Name:

University Mentor Name: Prof. Maya Baniya

Academic Year: 2026

Table of Contents

1. **Introduction**
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. **System Design**
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. LangChain Workflow
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. **Data Design**
 - 3.1 Data Model
 - 3.2 Data Access Mechanism
4. **Interfaces**
5. **State and Session Mangement**
6. **Non Functional Requirements**
 - 7.1 Security Aspects
 - 7.2 Performance Aspects
7. **References**

1. Introduction

1.1 Scope of the document

This document describes the low-level design of the MedScribe AI. It covers internal architecture, module-level design, data flow, API structure, Agentic working and implementation details required to build and maintain the system.

1.2 Intended Audience

- Developers
- Testers
- Project evaluators
- Technical reviewers

1.3 System Overview

The MedScribe AI is an Agentic-GenAI-based application designed to generate structured medical documents such as patient summaries and medical report and AI Agent that autonomously plans, validates resources, builds schedules, and replans when resources are unavailable.. The system uses Retrieval-Augmented Generation (RAG) by combining a vector database (ChromaDB) with a Large Language Model (Gemini API) and LangChain. It ensures consistent formatting, professional tone, and domain-specific terminology.

2. System Design

2.1 Application Design

The system follows a layered architecture consisting of three main layers: Frontend Layer, Backend Layer, and AI & Data Layer. It implements a Retrieval-Augmented Generation (RAG) pipeline to generate accurate and structured healthcare content.

Frontend Layer (User Interface)

- Built using HTML, CSS, and JavaScript
- Allows users (healthcare professionals) to input patient details
- Sends data to backend via HTTP POST (/generate)
- Dedicated button to switch to agent and use it for task like decomposing objective into actionable steps and also ensure resource validity.
- Displays the generated patient summary.

Backend Layer (Flask)

- Acts as the core processing unit
- Handles API requests through the API Controller (app.py)
- Manages the RAG pipeline workflow
- Coordinates between embedding model, vector database, and LLM

Agent Layer

- Provides goal to agent and then validates resource and reschedule tasks and steps if needed via langchain.
- **LangChain:**
 - AgentExecutor autonomously selects which @tool to call based on step description — no manual tool routing
 - Provides structured step-by-step procedure to be followed.

AI & Data Layer

This layer provides intelligence and context-aware generation:

- **Embedding Model (SentenceTransformer):**
 - Converts input text into vector embeddings
- **Vector Database (ChromaDB):**
 - Stores embeddings and generated documents
 - Retrieves similar past records for context (RAG)
- **LLM API (Gemini 2.5 Flash):**
 - Generates structured patient summaries
 - Uses both user input and retrieved context

Architecture:

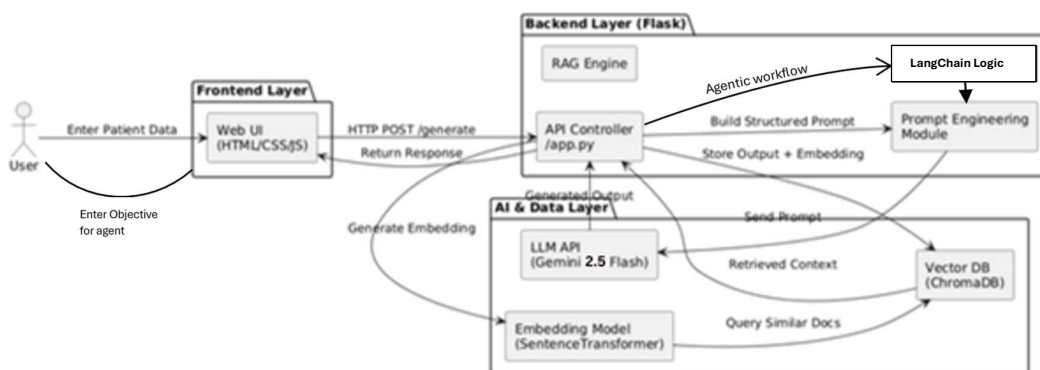
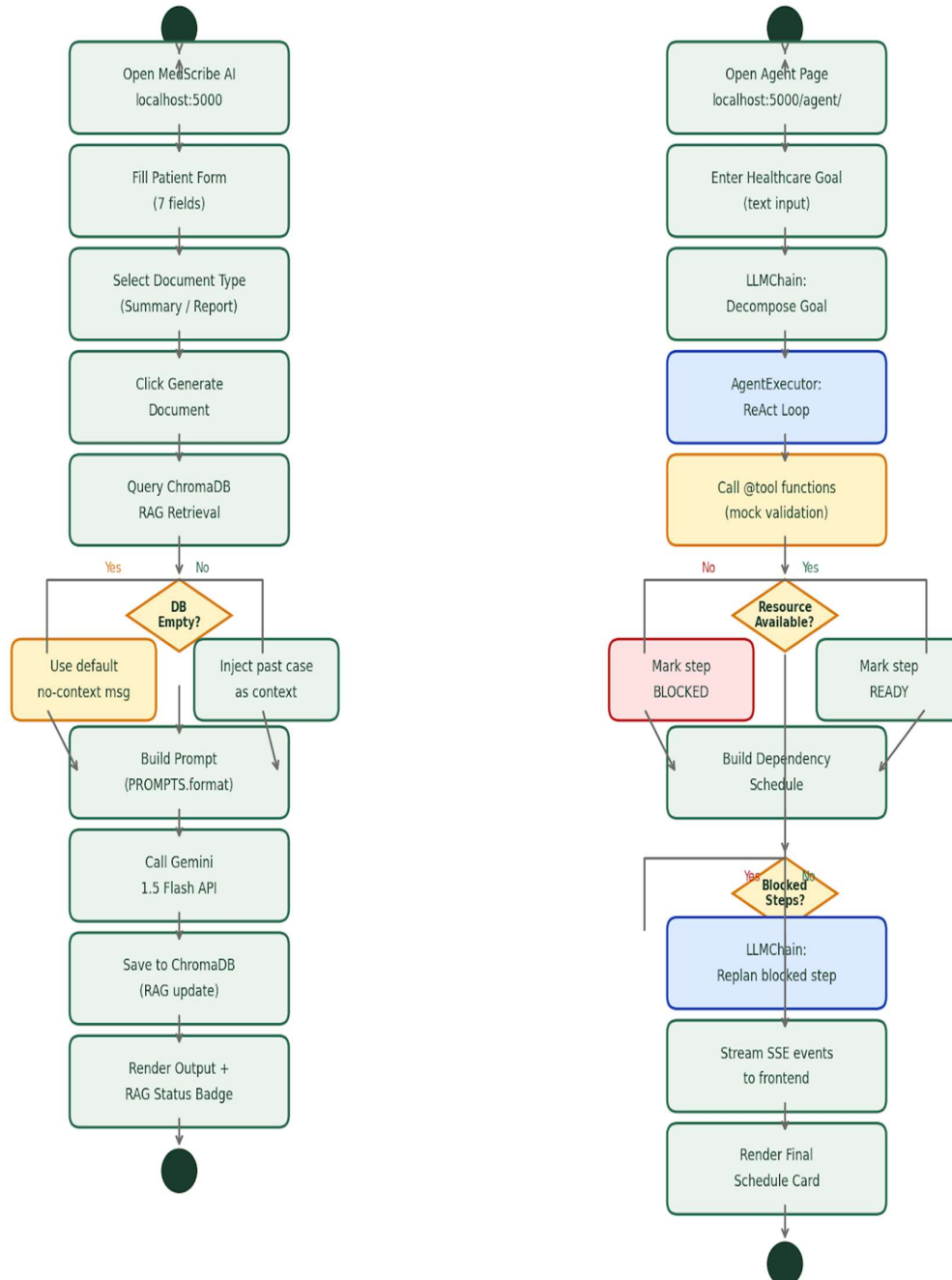


Fig. Architecture Diagram

2.2 Process Flow

Activity Diagram — Full Application Flow



Fig, Activity Diagram

Workflow Description (Agent):**User Input:**

The process starts when the user enters goal in the frontend.

Send to Backend:

The input is sent to the Flask backend via an API request.

LLM Chain:

The goal is then decomposed into small actionable steps. Also the prompt is part done here.

AgentExecutor and validation:

The ReAct Loop is activated and then the tools are validated with generated schedule.

Decision – Resource Availability?:

- **Yes:** Mark step ready and build dependency schedule
- **No:** Show steps blocked and build dependency schedule

Decision – Blocked Steps?:

- **Yes:** Server-Sent-Events to push real time data to frontend
- **No:** Replan blocked steps and then SSE to frontend

Output Generation:

The Final Schedule card is shown at frontend.

2.3 Information Flow

- Input Data: High-level goal or objective
- LLM Chain and @tools: Breaking down of steps and resource Validation
- Scheduling: Steps are generated and replan if needed and then finalize
- Output Data: Structured Schedule card

Flow:

User Input → LLM Chain → Tools → Steps → Finalize → Output

2.4 LangChain Workflow

File	LangChain Component	What It Does
agent/tools.py	@tool decorator	Converts 5 plain Python functions into LangChain BaseTool objects that AgentExecutor can call autonomously
agent/planner.py	ChatGoogleGenerativeAI	LangChain LLM wrapper for Gemini — compatible with LLMChain
agent/planner.py	PromptTemplate	Declares input_variables explicitly; validates them before formatting; integrates with LLMChain
agent/planner.py	LLMChain	Links PromptTemplate + LLM — chain.invoke({...}) handles formatting + LLM call in one step
agent/executor.py	ChatGoogleGenerativeAI	Separate LLM instance for the ReAct agent with temperature=0.1
agent/executor.py	PromptTemplate.from_template	ReAct prompt with required {tools}, {tool_names}, {input}, {agent_scratchpad} placeholders
agent/executor.py	create_react_agent	Builds the ReAct reasoning agent from llm + tools + prompt
agent/executor.py	AgentExecutor	Runs the autonomous think -> tool -> observe loop with max_iterations=4

2.5 Key Design Considerations

Accuracy: Use RAG to improve contextual relevance

Consistency: Enforce structured format (SOAP)

Scalability: Modular backend design

Security: Avoid sensitive data exposure

Maintainability: Clean separation of modules

Agentic working: Separate interface for cleaner usage and implementation

2.6 API Catalogue

Generate Content API

- Endpoint: /generate
- Method: POST

Request Body: Input via form and prompt

Response:

- Generated Summary
- Structured Schedule Card

3. Data Design

3.1 Data Model:

MedScribe AI follows a four-layer architecture. The Presentation Layer handles all browser-based user interaction. The Application Layer (Flask) handles routing and orchestration. The Agent Layer implements the LangChain-powered planning pipeline and helps to provide the agentic capabilities. The Data and AI Layer contains ChromaDB, Gemini API, and LangChain component instances. The modular architecture helps to understand. It also helps to troubleshoot and add newer capabilities and promotes accessibility.

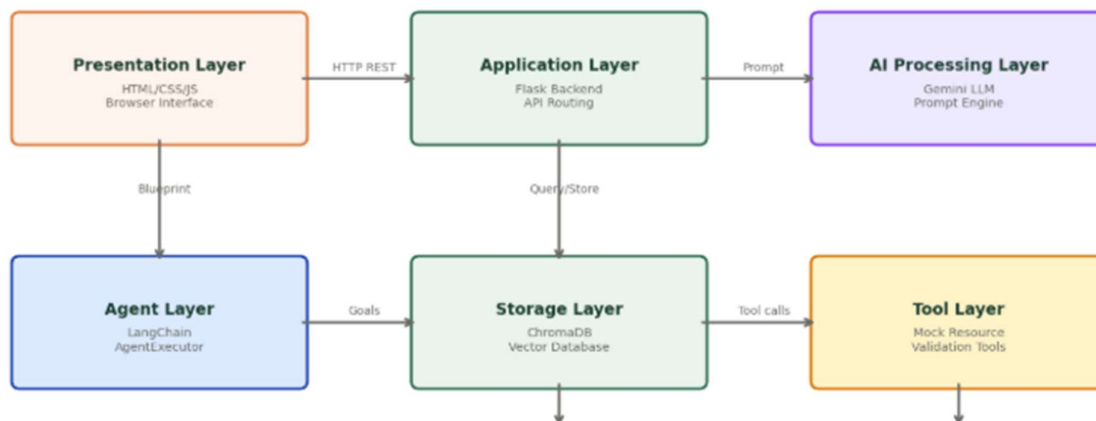


Fig. Component Diagram

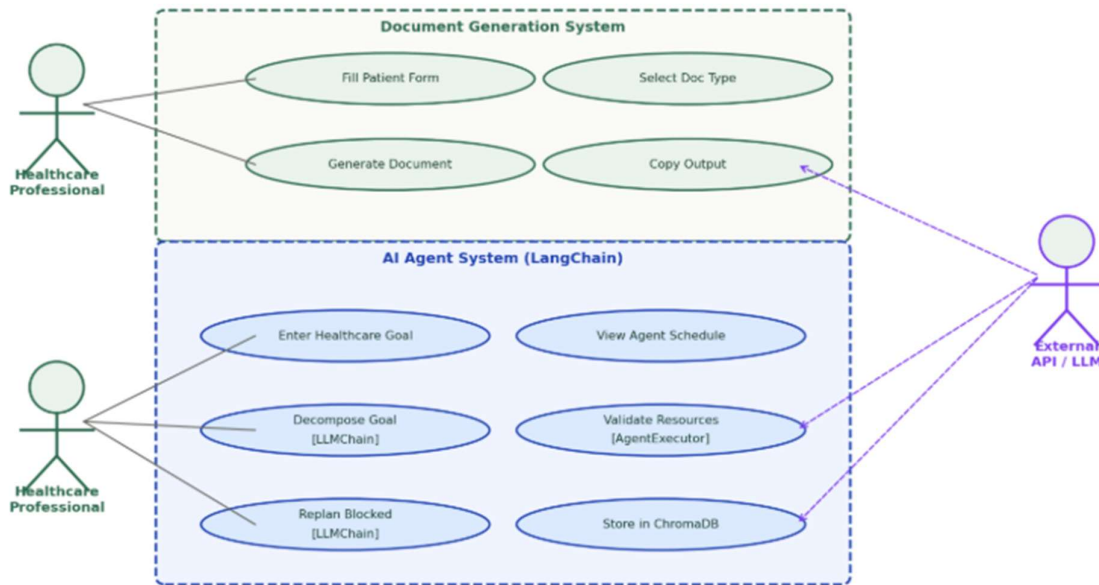


Fig. Use Case Diagram

3.2 Data Access Mechanism

For Document Generation: User fills data in patient form and select document type(Summary or medical report) and then browser sends an HTTP POST request to backend, the backend then take the data and provide it to the prompt and then it goes to the Gemini 2.5 Flash model with vector embeddings and chromaDB usage and after checking for previous matching queries and then the Generated output will again be provided to the frontend interface.

Agent Pipeline Flow: User enters healthcare goal on /agent/ page. Browser opens SSE connection and Flask streams SSE events as each phase completes. LangChain LLMChain decomposes goal into 5-8 structured steps. LangChain AgentExecutor runs ReAct loop — calls @tool functions for each step. Dependency-aware schedule built from step results. LangChain LLMChain replans any blocked steps via Gemini. Final schedule card rendered on frontend after checking the tools and availability and modification resulting in a well structure procedure.

4. Interface

AI Agent

MedScribe AI

CLINICAL DATA ASSISTANT

Generate Clinical Document

Fill in the patient details below to generate a professional healthcare document.

Patient Summary

Medical Report

PATIENT NAME *

e.g. Viraj Agrawal

AGE *

e.g. 25

GENDER

Male

CURRENT MEDICATIONS

e.g. Paracetamol 500mg

CHIEF COMPLAINT *

e.g. Recurring dizziness, mild hypertension

MEDICAL HISTORY

e.g. Type 2 diabetes (diagnosed 2018), hypertension, no prior surgeries

ADDITIONAL NOTES

Any other relevant information...

Generate Document

Homepage

EXAMPLE GOALS

Find treatment options for Type 2 Diabetes

Create a cardiac rehabilitation plan for a post-MI patient

Develop a cancer screening protocol for high-risk patients

Plan management for a patient with chronic kidney disease

Organise a complete diagnostic workup for unexplained weight loss

AGENT PIPELINE

- Decompose goal into steps
- Validate each resource
- Build dependency schedule
- Replan blocked steps

STEP CATEGORIES

- Research
- Specialist
- Diagnostic
- Insurance
- Treatment
- Follow-up

MOCK TOOLS

- check_specialist
- check_equipment
- check_insurance
- get_patient_history
- estimate_duration

\$

Healthcare AI Agent

Enter a broad healthcare goal. The agent will decompose it into steps, validate resources, build a dependency-aware schedule, and adapt to any unavailable resources.

Agent Interface

5. State and Session Management

It is designed with a lightweight, stateless architecture that prioritizes speed and data privacy. The app treats every request as a fresh start, making it highly efficient. However, it manages state through **Retrieval-Augmented Generation (RAG)**. On the front end, simple JavaScript keeps track of what the user is currently typing and which document type they've selected. On the back end, the system uses **ChromaDB** as a sort of "long-term memory". When a user submits a case, the app quickly looks up similar past entries in this vector database to give the AI the right clinical context. The LangChain architecture helps to manage the working of agent in an efficient manner. While it currently runs on a fast, in-memory cache that resets with the server, it's built to easily switch to permanent storage whenever needed.

6. Non Functional Requirements

Performance: The system utilizes **ChromaDB** as an in-memory vector database to ensure near-instant retrieval of similar clinical contexts (RAG) during the generation process.

Scalability: The backend is built using **Flask**, a lightweight WSGI web application framework that allows for easy deployment.

Reliability: By integrating the **Gemini 2.5 Flash** model, the system leverages a high-availability cloud LLM API to ensure consistent document generation performance.

Usability: The frontend is designed with a clean, professional healthcare-oriented UI using **Vanilla JS, CSS, and HTML**, featuring a "one-click" generation pipeline and clear document type selectors.

Robustness: The system must handle tool failures gracefully. If a scheduling tool is down, the agent should alert the user rather than failing silently.

Maintainability: the project follows a modular structure, separating the logic into a main backend (app.py), a clear dependency list (requirements.txt), and organized static assets (index.html).

7. References

Google Gemini API Documentation — <https://ai.google.dev>

ChromaDB Documentation — <https://docs.trychroma.com>

Flask Documentation — <https://flask.palletsprojects.com>

Prompt Engineering Guide — <https://wmv.promptingguide.ai>

LangChain Documentation — <https://python.langchain.com>